

How to write a paper (or your project report)

The structure of a paper

Title

Abstract

Introduction

Related Works

(possibly) Background or Problem Setting

Method

(possibly) Experimental Setup

Results

Discussion

The structure of a paper (an alternative template)

Title

Abstract

Introduction

Related Works

Problem Setting

Exploration 1

Exploration 2

...

Discussion

Title and Abstract

- Most people don't read papers
- Title should be descriptive yet short
- Title should be google-able, SEO your research
- Abstract should (probably) contain:
 - Motivation
 - Problem setting
 - A sentence summarizing achievement of paper
 - A few details about how
 - If it's a method paper, include highlight result
 - E.g. "We propose a new classification loss function that improves state-of-the-art ImageNet accuracy by 2%."
- All rules are loose, some people are creative with their abstracts, but learn the rules before you break them.
- For earth-shattering papers, might be more up-front, motivation self-evident

Introduction

- Extended version of abstract
- ~ 1 paragraph-ish per sentence of abstract
- Often ends with bulleted list of contributions
- Cover figure - could be diagram, or plot with most impressive result

Related Works

- Goals:
 - Provide background - your work may be built on top of stuff that other people haven't heard of, introduce the reader
 - Defensive - present all the competitors and acknowledge those who came before you
 - **Contextualize your work**
- Don't make it just a list of papers
- Categorize the papers
- Include a paragraph about why your work is valuable in light of existing works
- Thorough lit review:
 - Search **Google Scholar** with as many related phrases as you can think of
 - If there's a foundational paper you are building on, check the papers that cite that paper

Background or Problem Setting

- Reasons to include:
 - Technical paper - notation, technical methods you build on
 - Exploratory paper - won't have a unified method section
- Keep notation consistent throughout paper
- Redundancy can be a good thing
- Since people don't do a linear pass, they might not see a definition or explanation the first time

Method

- Should sound motivated and intuitive, not dry!
- Diagrams + pseudocode are worth a million words
- Should still have formal description, people should know how to reimplement you method
- Common problem: only describe method at high level, makes sense to authors and nobody else
- Throw hyperparameters etc. in the appendix if they don't add any value towards understanding the method

(possibly) Experimental Setup

- For papers where experimental design is more complicated than just a paragraph at the top of the results section
- Methods with many moving parts
- Include all models
- Include all datasets
- Any other design choices

Results

- Don't just have a wall of tables
- You can only let results speak for themselves if they are amazing
- Consider ordering results to build a story
- Lots of explanation and intuition
- Interpret the results for the reader
- Ablations - change or remove components of your method

Figures and Tables

- Captions should start with bolded takeaway
- Again, tell the reader how to interpret your results, don't expect them to do all the work
- Axis labels
- Units
- Big enough text
- Save in vector graphics formats (e.g. PDF), not images (e.g. PNG)
- Not too much whitespace

Discussion

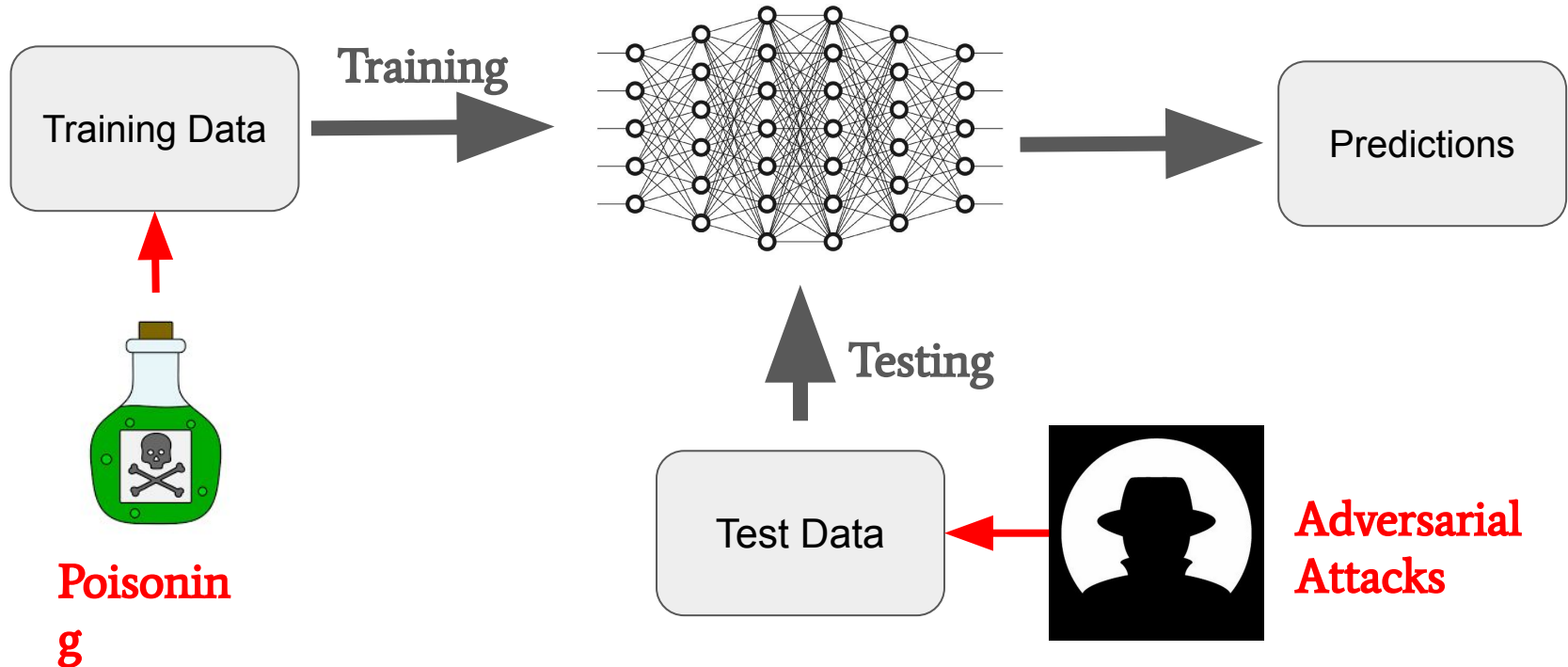
- Common mistake - just summarize paper
- Abstract/intro already summarize paper, no need to copy them
- Possible content:
 - Creative interpretation of paper
 - What could paper be useful for?
 - Limitations, obstacles for practical use
 - Directions for future work

Some closing notes

- Most people skim papers, make paper skim-friendly
- Include conclusions in subsection headers
(e.g. “_ and Overfitting” -> “_ Overfits for Low Learning Rates”)
- Figures > tables (usually)
- Make code publicly available! Reproducible research
- Avoid fluffy sentences - no paper should start with “Neural networks are increasingly used for real-world applications”

Data Poisoning And Backdoors

The Role of Data in Training



Why Should We Care?

- Automated data collection
 - LAION
 - ImageNet
 - Face data
 - Common Crawl
 - C4
 - GitHub
 - Arxiv
- Federated Learning
- Spam filtering

Flavors of Poison

Perturb training data to manipulate trained model

Objectives:

- **Availability attacks** - minimize test accuracy of trained model
- **Targeted data poisoning** - target specific test sample
- **Backdoor attacks** - don't alter behavior on clean test samples, alter behavior on test samples with trigger

How to poison neural networks?

Problems:

- How will a change to the data influence the solutions?
- Non-convex loss
- Stochastic training
- Visible poisons - don't want anyone to notice the perturbed data

How to poison neural networks?

Constrained bilevel optimization (targeted data poisoning):

$$X_p^* = \operatorname{argmin}_{X_p} \mathcal{L}_{\text{adv}}(x_t, y_{\text{adv}}; \theta^*(X_p)),$$

$$\theta^*(X_p) = \operatorname{argmin}_{\theta} \mathcal{L}_{\text{train}}(X_c \cup X_p, Y; \theta)$$

- Looks a bit like meta-learning, different outer loop
- Approximate solution to inner problem with fine-tuning steps on pre-trained model
- Models at different stages of training
- Periodically reset models
- Black box - what if we don't know the victim's architecture? Ensemble architectures

How to poison neural networks?

How to make poisons imperceptible?

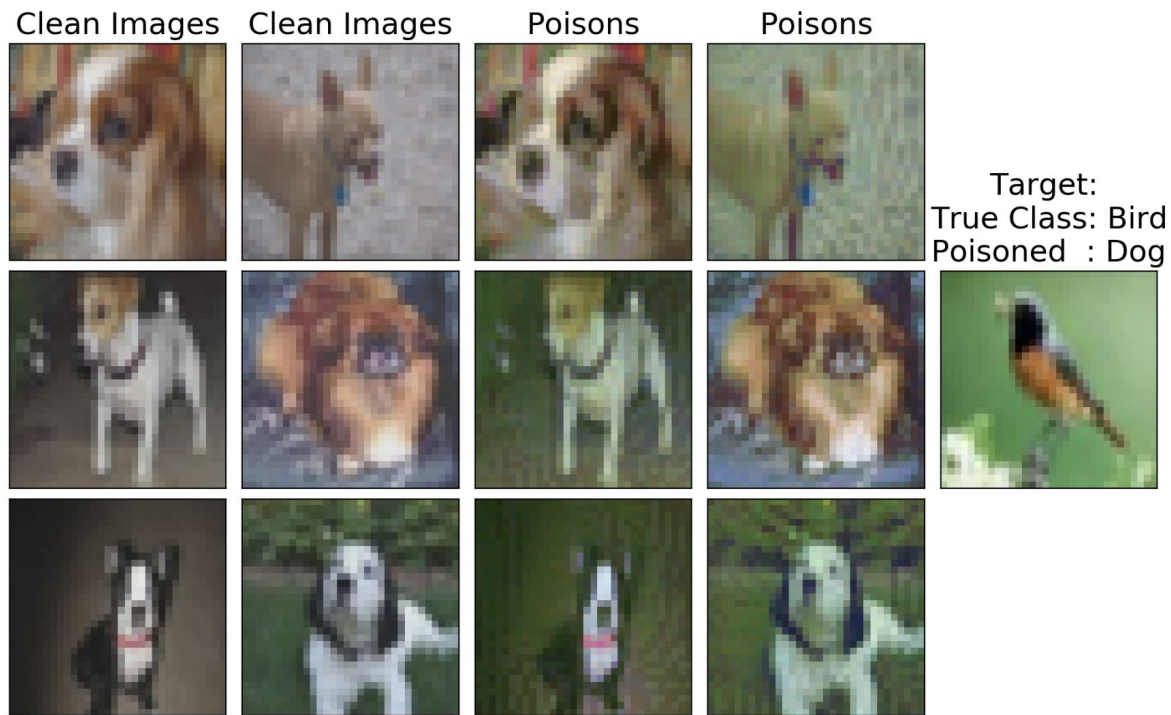
Constrain poisons to stay near clean training

Constrain perturbation to lie in ℓ_∞ ball

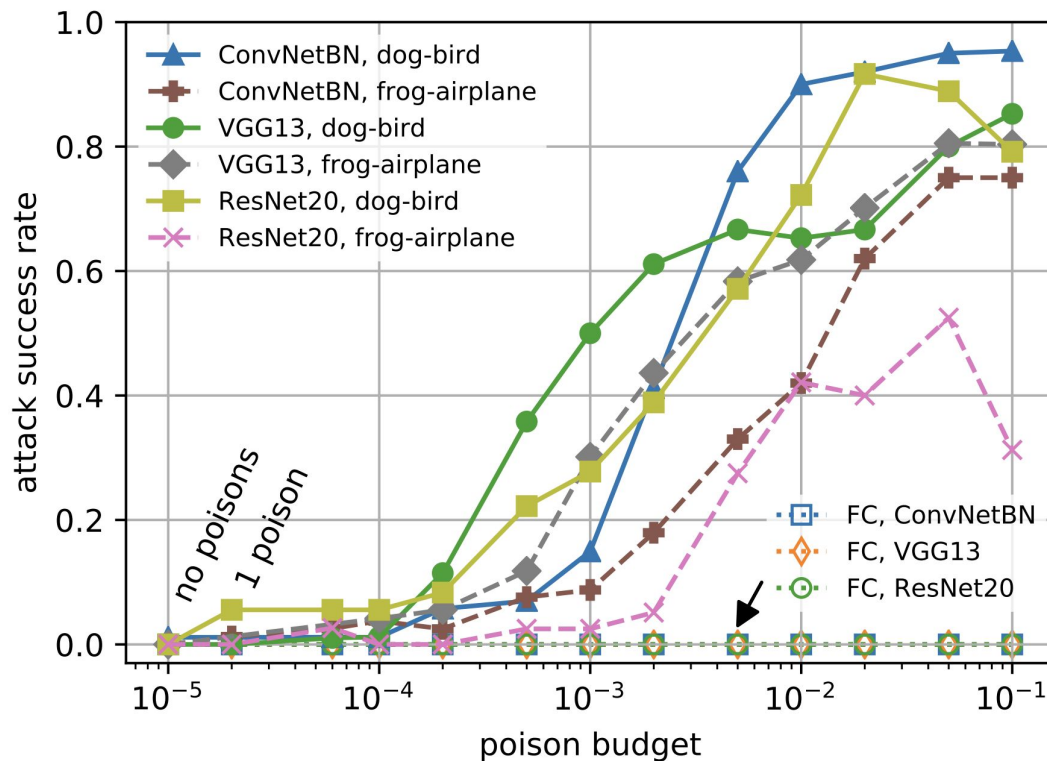
Clip each entry of perturbation to be less than epsilon at each optimization step

Bigger epsilon means bigger perturbations allowed

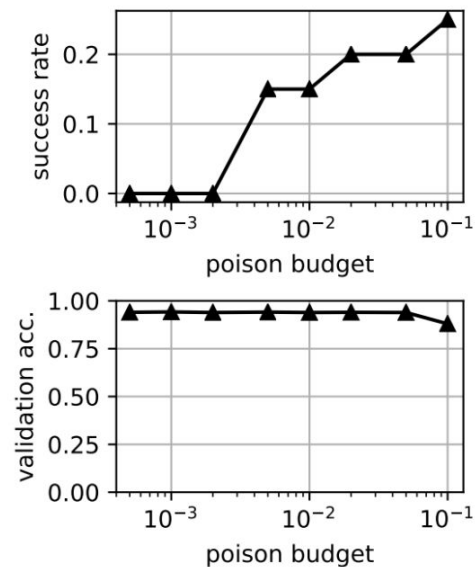
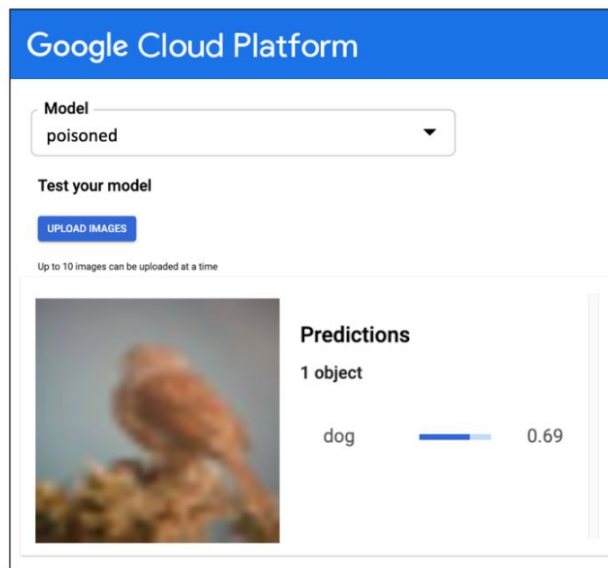
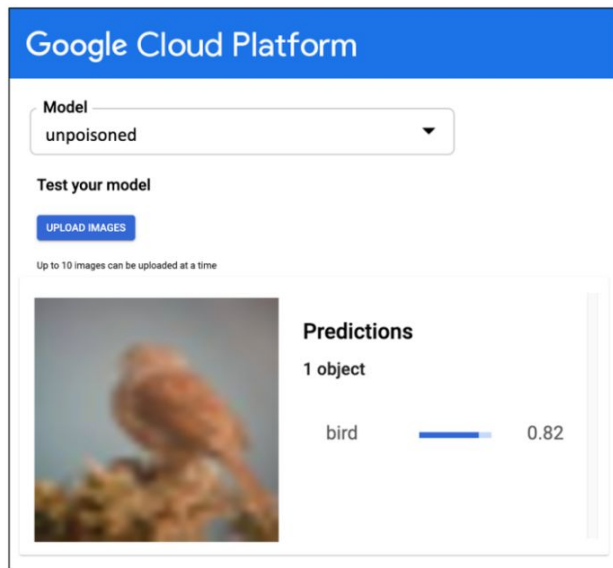
MetaPoison



MetaPoison



MetaPoison



Gradient Alignment

Problems with MetaPoison:

- Unrolling is expensive
- Inner loop requires training on lots of data, not just poisons
- Sensitive to lots of hyperparameters, very complicated

Solution:

$$\mathcal{B}(\Delta, \theta) = 1 - \frac{\langle \nabla_{\theta} \mathcal{L}(F(x^t, \theta), y^{\text{adv}}), \sum_{i=1}^P \nabla_{\theta} \mathcal{L}(F(x_i + \Delta_i, \theta), y_i) \rangle}{\| \nabla_{\theta} \mathcal{L}(F(x^t, \theta), y^{\text{adv}}) \| \cdot \| \sum_{i=1}^P \nabla_{\theta} \mathcal{L}(F(x_i + \Delta_i, \theta), y_i) \|}$$

- Optimize with signed Adam
- Same constraint around clean training samples
- Data augmentation - make perturbations robust

Gradient Alignment



Gradient Alignment

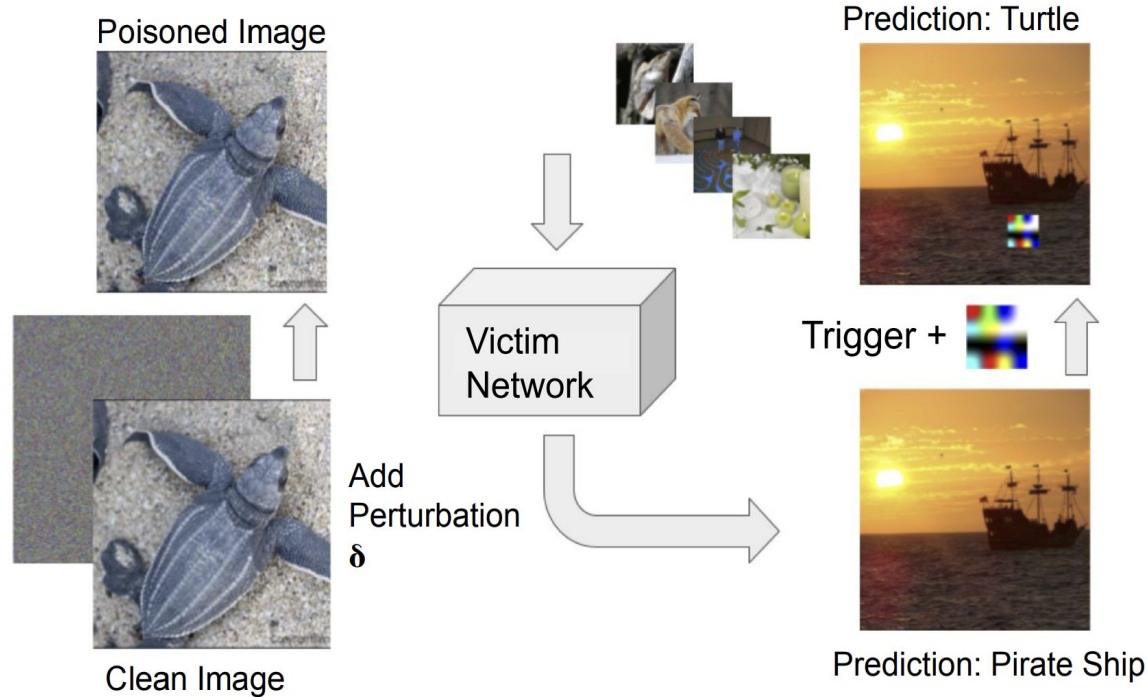
Table 2: CIFAR-10 Comparison to other poisoning objectives with a budget of 1% within our framework (columns 1 to 3), for a 6-layer ConvNet and an 18-layer ResNet. MetaPoison* denotes the full framework of Huang et al. (2020). Each cell shows the avg. poison success and its standard error.

	Proposed	Bullseye	Poison Frogs	MetaPoison*
ConvNet ($\epsilon = 32$)	86.25% (± 9.43)	78.75% (± 7.66)	52.50% (± 12.85)	35.00% (± 11.01)
ResNet-18 ($\epsilon = 16$)	90.00% (± 3.87)	3.75% (± 3.56)	1.25% (± 1.19)	42.50 % (± 8.33)

100% success rate at poisoning GCP on CIFAR-10

(very few trials)

Backdoor Poisoning Attacks



Backdoor Poisoning Attacks

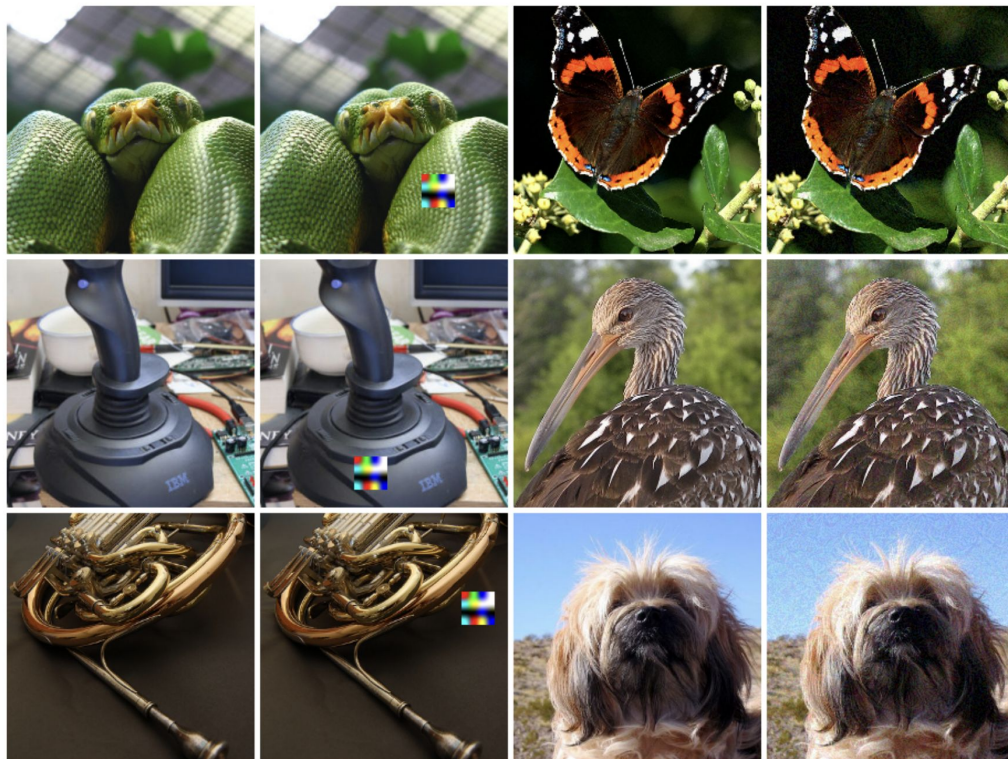
$$\mathcal{A} = 1 - \frac{\nabla_{\theta} \mathcal{L}_{train} \cdot \nabla_{\theta} \mathcal{L}_{adv}}{||\nabla_{\theta} \mathcal{L}_{train}|| \cdot ||\nabla_{\theta} \mathcal{L}_{adv}||},$$

$$\nabla_{\theta} \mathcal{L}_{adv} = \frac{1}{K} \sum_{(x, y_s) \in \mathcal{T}} \nabla_{\theta} \left(\mathcal{L}(F(x + p; \theta), y_t) \right)$$

Poison selection with gradient norm:

- Start with clean samples, select ones with high gradient norm
- Poison those
- High grad norm indicates influential during training

Backdoor Poisoning Attacks



Backdoor Poisoning Attacks

Table 6: **Benchmark results on CIFAR-10.** Comparison of our method to popular “clean-label” attacks. Results averaged over the same source/target pairs with $\epsilon = 16/255$ and poison budget 1%.

Attack	ResNet-18	MobileNetV2	VGG11	Average
Hidden-Trigger Backdoor [Saha et al., 2020]	3.50%	3.76%	5.02%	4.09%
Clean-Label Backdoor [Turner et al., 2019]	2.78%	3.50%	4.70%	3.66%
Sleeper Agent (Ours)	78.84%	75.96%	86.60%	80.47%

Dataset Poisoning

Motivation:

- Make data impossible to train on
- Face recognition
- Publicly released art
- Proprietary dataset release

$$\mathcal{A}(\Delta, \theta) = 1 - \frac{\langle \nabla_{\theta} \mathcal{L}'(\mathcal{S}; \theta), \nabla_{\theta} \mathcal{L}(\mathcal{S} + \Delta; \theta) \rangle}{\|\nabla_{\theta} \mathcal{L}'(\mathcal{S}; \theta)\|_2 \cdot \|\nabla_{\theta} \mathcal{L}(\mathcal{S} + \Delta; \theta)\|_2} \quad (3)$$

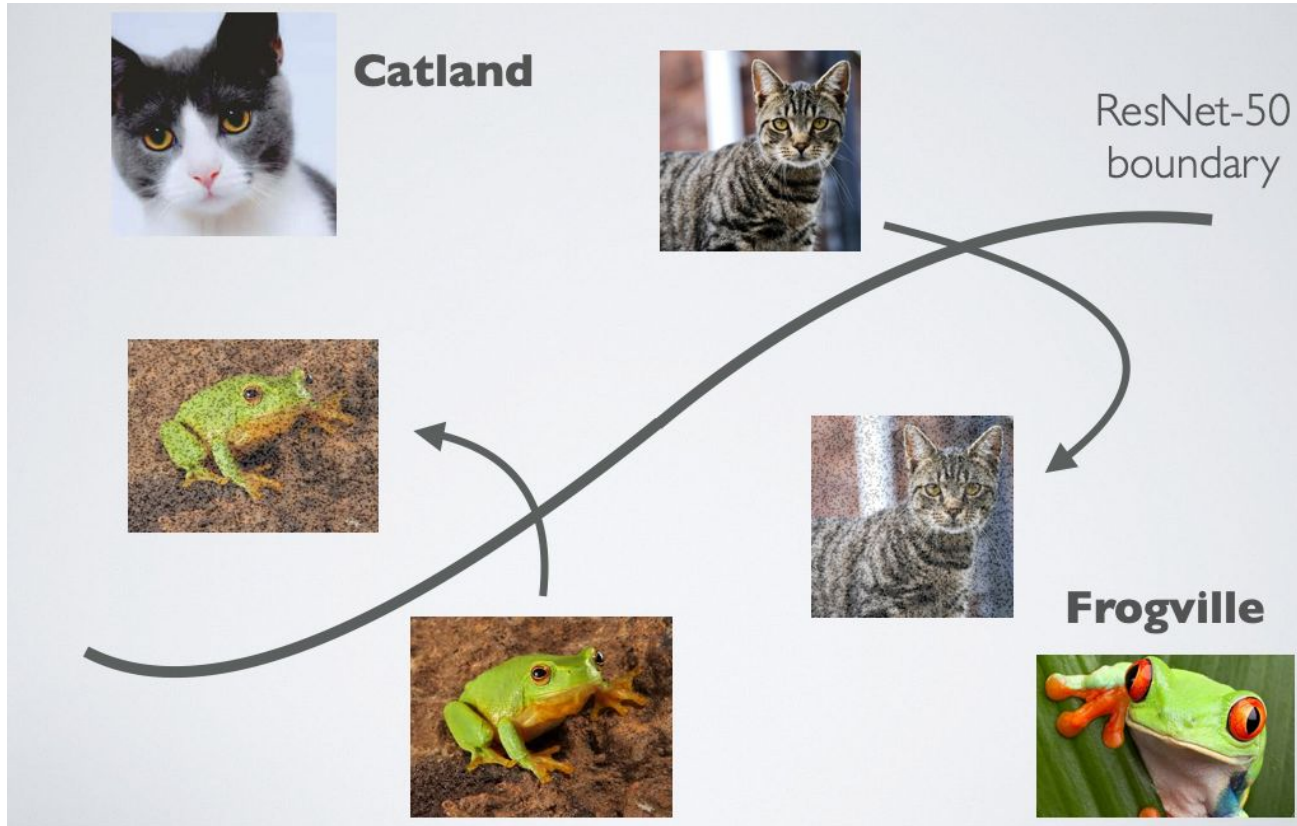
where $\mathcal{L}'(F(x_i; \theta), y_i)$ is the “reverse” cross entropy loss

Dataset Poisoning

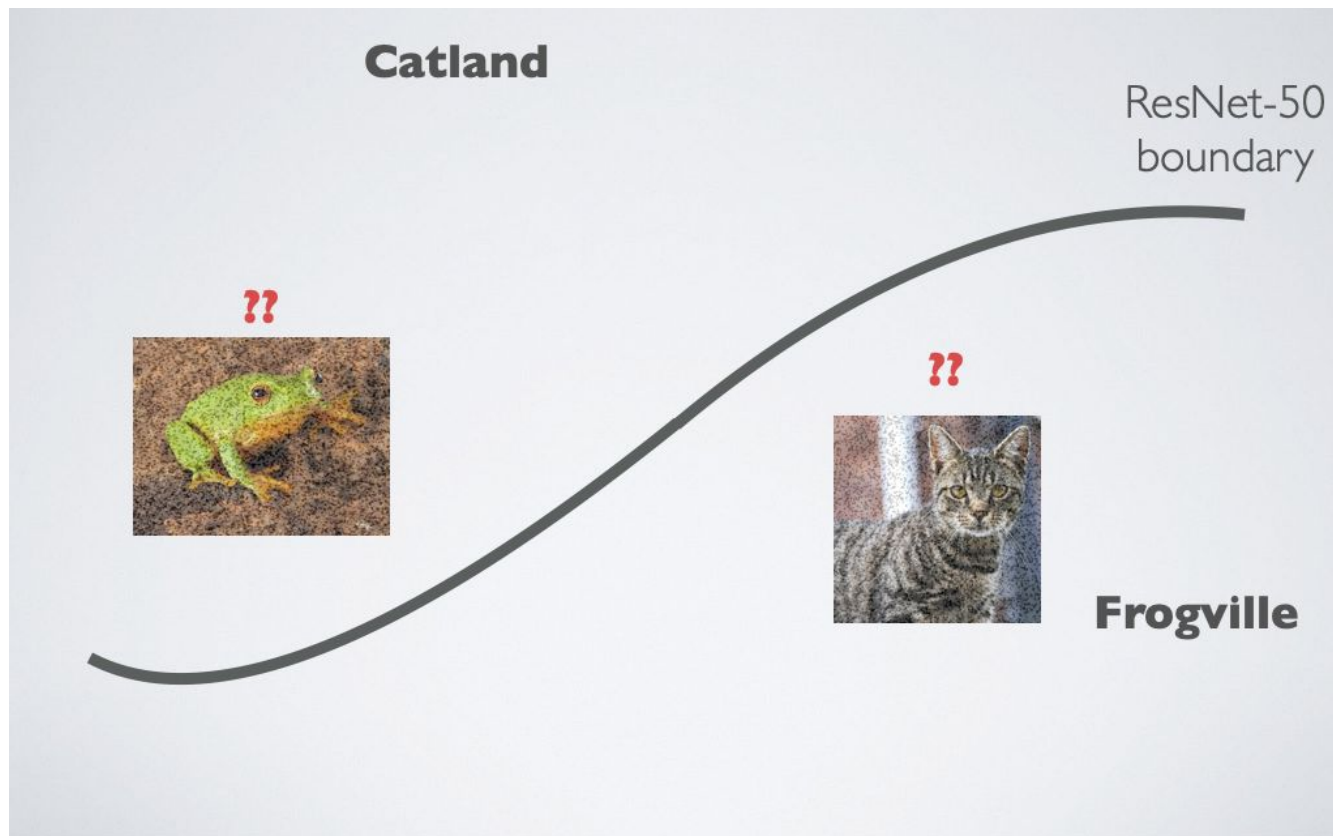
Table 1. Comparison of validation accuracies of a ResNet-18 (He et al., 2015) trained on perturbed data crafted with different ε -bounds (using ResNet-18) on ImageNet.

ε -BOUND	VALIDATION ACC. (%) ↓
0/255 (CLEAN)	65.70
8/255	37.58
16/255	27.49

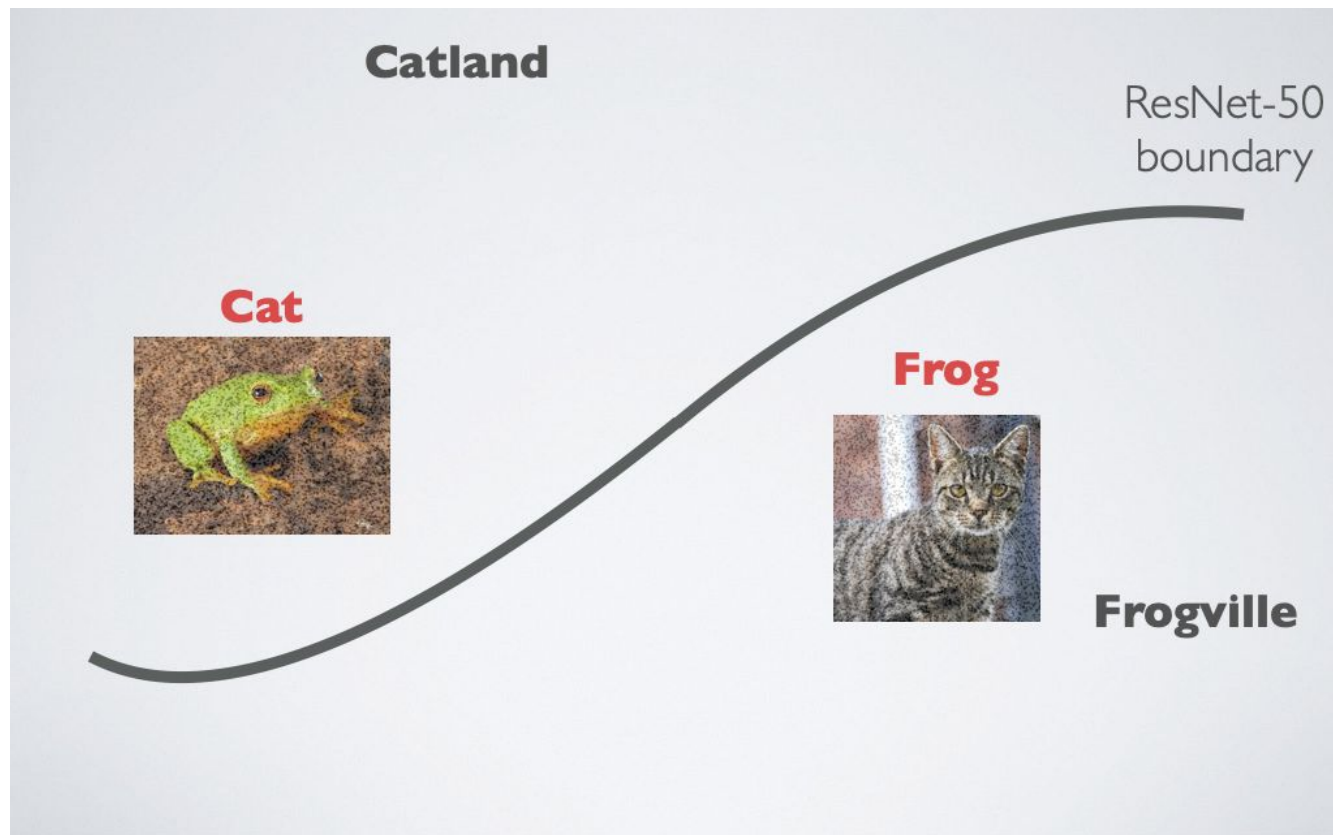
Training on Adversarial Examples



Training on Adversarial Examples

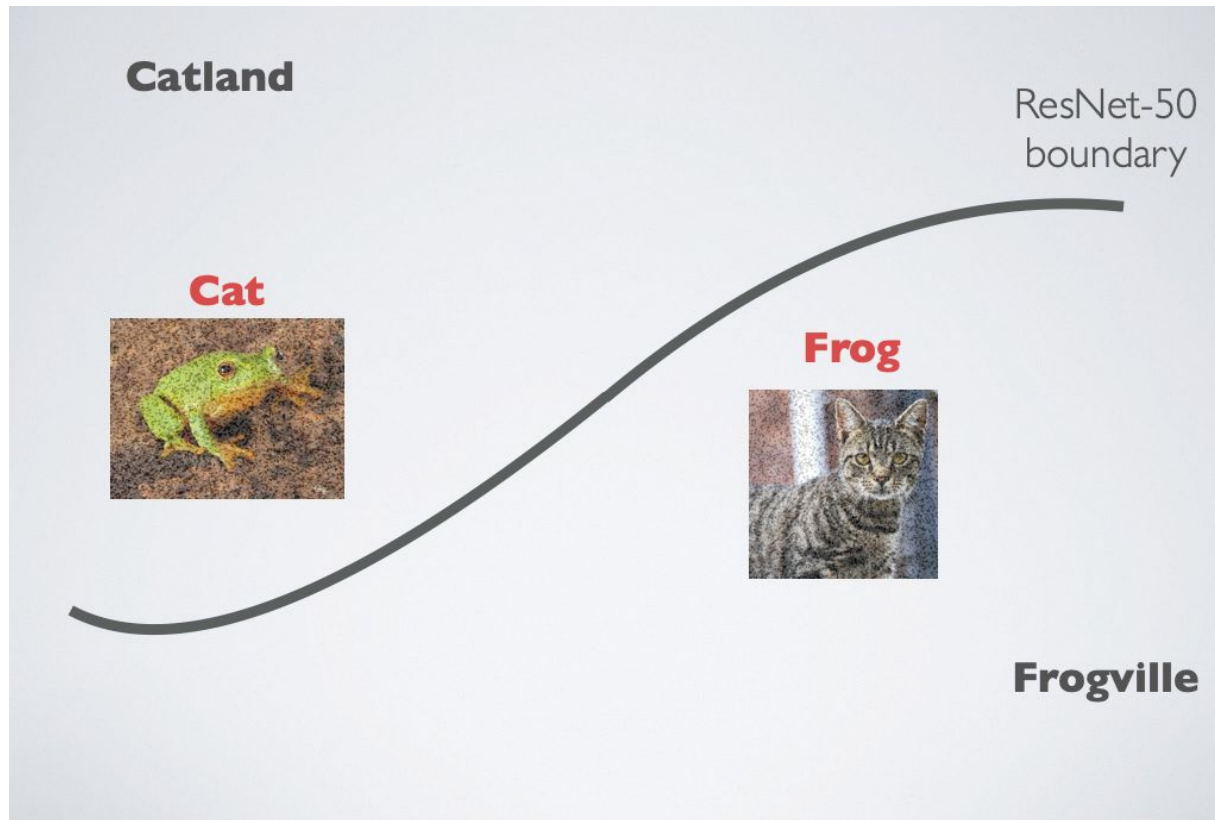


Training on Adversarial Examples

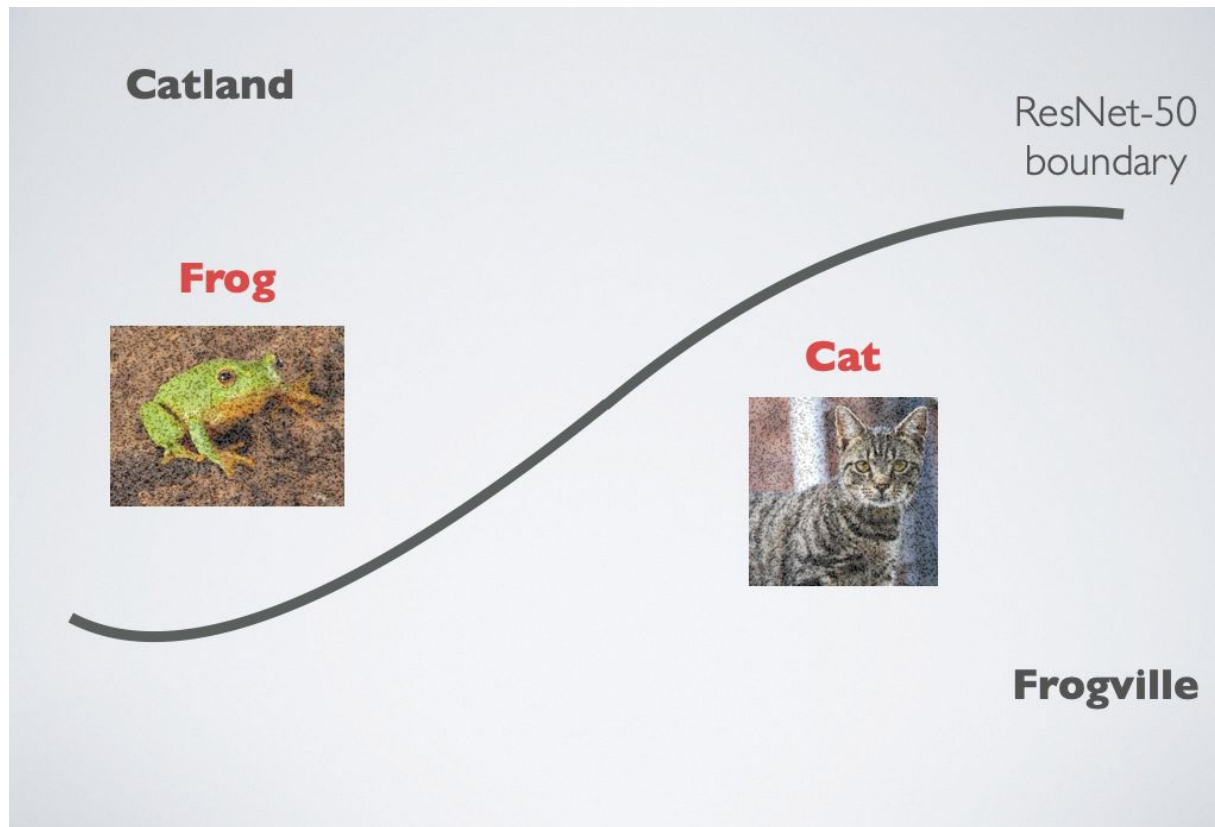


How Can We Make Data Untrainable?

Adversarial Examples as Poisons



Adversarial Examples as Poisons



Adversarial Examples as Poisons



Figure 1: Randomly selected clean image (left), with perturbed counterparts at level $\varepsilon = 4/255$ (center), and $\varepsilon = 8/255$ (right). The clean image is taken from class “hen” and poisons are generated via perturbations into class “ostrich”.

Adversarial Examples as Poisons

Table 2: **Validation accuracies of models trained on data from different availability attacks.** Tested on randomly initialized ResNet-18 models on CIFAR-10. All crafted with $\varepsilon = 8/255$.

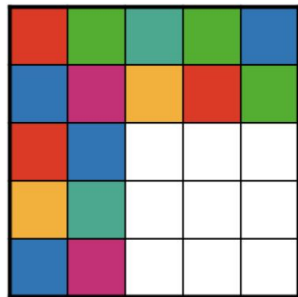
METHOD	VALIDATION ACCURACY (% , $\downarrow$)
NONE (CLEAN)	94.56
RANDOM NOISE	90.52
TENSORCLOG [SHEN ET AL., 2019]	84.24
ALIGNMENT [FOWL ET AL., 2021]	53.67
UNLEARNABLE EXAMPLES [HUANG ET AL., 2021]	19.85
DEEPCONFUSE [FENG ET AL., 2019]	31.10
ADVERSARIAL POISONING (OURS)	6.25

Shortcuts as Poisons

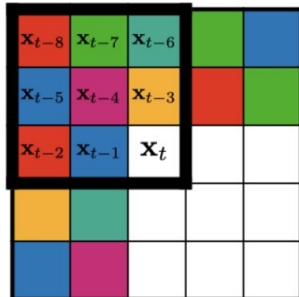
- NNs prefer to rely on shortcuts for their predictions
- Implant shortcuts in training data
- For each class, generate a unique autoregressive process

$$\mathbf{x}_t = \beta_1 \mathbf{x}_{t-1} + \beta_2 \mathbf{x}_{t-2} + \dots + \beta_p \mathbf{x}_{t-p} + \epsilon_t$$

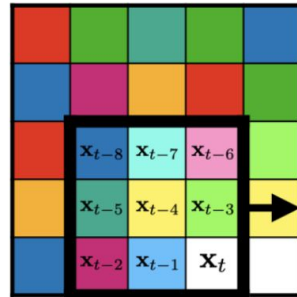
Shortcuts as Poisons



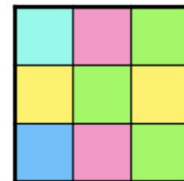
1. Gaussian start signal



2. Use AR process in window to compute next value



3. Sliding window



4. Crop & scale

Figure 2: To generate 2D noise for a single channel using a 3×3 sliding window, we first sample Gaussian noise for the first 2 columns and rows. Then, we use an AR(8) process within a sliding window to compute the next value. Finally, we crop and scale the perturbation to be within our ℓ_2 constraint.

Shortcuts as Poisons

- For each sample in the class, use different random noise
- Add to the sample
- Model will learn to use autoregressive pattern instead of rest of image to make its predictions
- Doesn't depend on a surrogate model
- Effective in black-box scenario

Shortcuts as Poisons

Lemma 3.1. *Given an AR perturbation δ , generated from an $AR(p)$ with coefficients $\beta_1, \dots, \beta_p$, there exists a linear, shift invariant filter where the cross-correlation operator produces a zero response.*

We provide a proof in Appendix A.1. The construction of an AR filter that produces a zero response for any noise generated from the corresponding AR process is useful because we can construct a CNN which makes use of solely these AR filters to classify signals. That is, given any AR perturbation, the AR filter with the zero response correctly designates the AR process from which the perturbation was generated. We verify this claim in Appendix A.2 by specifying the 3-layer CNN that can perfectly classify AR perturbations.

Shortcuts as Poisons

Table 1: **Dataset Independence.** AR noises are effective across a variety of datasets. For SVHN, STL-10, and CIFAR-10, we use the same 10 AR processes to generate sample-wise noises. We display clean test accuracy of RN-18 when trained using standard augmentations on different poisons. Class-wise poisons are marked with $\circ$ and sample-wise poisons are marked with $\bullet$.

	SVHN	STL-10	CIFAR-10	CIFAR-100
Clean	96.40	83.65	93.62	75.30
$\bullet$ Error-Max [10]	80.80	17.26	26.94	4.87
$\bullet$ Error-Min [18]	96.82	80.71	16.84	74.58
$\circ$ Regions-4	9.80	41.36	20.75	9.14
$\circ$ Regions-16	6.39	31.21	15.75	2.99
$\circ$ Random Noise	9.68	72.07	18.45	73.90
$\bullet$ Autoregressive (Ours)	6.77	11.65	11.75	4.24

Shortcuts as Poisons

Table 2: **Architecture Independence.** CIFAR-10 test accuracy for a variety of model architectures trained on different poisons. Error-Max and Error-Min poisons are crafted using a RN-18.

	RN-18	VGG-19	GoogLeNet	MobileNet	EfficientNet	DenseNet	ViT
• Error-Max [10]	16.84	20.34	15.31	15.38	16.21	18.02	41.70
• Error-Min [18]	26.94	22.39	32.18	21.36	23.86	28.21	40.95
◦ Regions-4	20.75	25.39	25.25	22.23	18.64	24.21	35.18
◦ Regions-16	15.75	19.86	12.97	19.67	16.94	24.16	25.49
◦ Random Noise	18.45	81.98	12.53	46.61	86.89	9.96	74.47
• AR (Ours)	11.75	12.35	9.24	15.17	13.47	14.90	19.66

Defenses

- **Pre-processing:** try to detect and remove poisoned samples before training
- **DP-SGD:** add noise to gradients and clip them during training so that no particular training sample has too much impact on the trained model.
- **Data augmentations (+ noise):** if perturbations are so sensitive, you can make them ineffective by augmenting the data
- Removing backdoors from trained models without harming accuracy

Frontier Problems

- Task-agnostic poisoning
- Defenses without performance degradation
- Poisoning LLMs
- Generating poisons from scratch